The Journal of Open Source Software

StatMerge: unified statistical analysis and inter-rater reliability for research

Mirza Niaz Zaman Elin [1]

1 AMAL Youth and Family Centre, St. John's, NL, Canada

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 07 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

StatMerge is a consolidated Python package and desktop application that brings together day–to–day research statistics with specialist routines for inter-rater agreement and reliability. It merges two previously independent code bases into a coherent library and GUI so that researchers, students, and practitioners can compute common effect sizes, confusion-matrix-based diagnostics, ROC/PR summaries, and agreement statistics such as Fleiss' kappa and intraclass correlation coefficients (ICC) within a single, documented tool. The package emphasizes pragmatic reproducibility: every algorithm is accessible from the Python API for scripted workflows and also from a minimal cross-platform GUI for teaching, demonstrations, and quick exploratory data analysis. Where possible, StatMerge provides references to standard definitions and implements widely used small-sample corrections and confidence intervals, such as Hedges' (g) and Wilson intervals for binomial proportions (Hedges & Olkin, 1985; Wilson, 1927).

Statement of need

Many applied research projects rely on a scattered collection of small scripts to perform tasks like computing Cohen's (d), checking model sensitivity/specificity from a table of predictions, or measuring the agreement of multiple annotators on categorical labels(Elin, 2025). These scripts are often ad hoc, inconsistently documented, and hard to reuse or validate. Existing libraries cover parts of this space—e.g., scikit-learn for ROC/PR utilities (Pedregosa & others, 2011), pandas for data handling (McKinney, 2010), and domain-specific packages for reliability—but there is value in a cohesive, light-weight toolkit that puts the most common analyses and agreement metrics behind a stable API (Haghighi et al., 2018; Icazatti et al., 2023; Kelley, 2021; Legrand & Allen, 2022; Patil, 2021) , accompanies them with a small GUI, and ships with tests and examples. StatMerge aims to meet that need. It reduces friction for researchers who switch between notebook-based computation and quick desktop exploration, and it offers a compact entry point for teaching methodological concepts such as effect sizes, rating agreement, and diagnostic trade-offs on ROC vs. precision–recall curves (Davis & Goadrich, 2006; Fawcett, 2006).

Design and scope

The package follows a simple structure under `src/statmerge`. The `analysis` module provides general-purpose effect sizes (Cohen's (d) and Hedges' (g)), deliberately implemented with transparent formulas to ease review and teaching (Cohen, 1988; Hedges & Olkin, 1985). The `metrics_mod` module focuses on agreement and diagnostic utilities: summary measures derived from confusion counts (accuracy, sensitivity/TPR, specificity/TNR, PPV/NPV, (F_1), balanced accuracy, Youden's (J), and Matthews correlation coefficient), plus binomial Wilson

confidence intervals for proportions (Wilson, 1927). It also includes helpers to generate ROC and precision–recall points and to compute area-under-curve summaries using the scikit-learn reference implementations when available (Pedregosa & others, 2011). For rater agreement, StatMerge implements Fleiss' kappa from raw label tallies for multiple raters (Fleiss, 1971) and the ICC(2,1) consistency model for continuous ratings, with an optional bootstrap to approximate confidence intervals (Koo & Li, 2016; Shrout & Fleiss, 1979).

A small Qt-based desktop application (PySide6) exposes these capabilities to non-programmers. The GUI bundles a "Data Lab" panel that can load a CSV, preview and describe columns, compute simple correlations, render quick plots via Matplotlib (Hunter, 2007), and export a short DOCX report. Two buttons launch the legacy graphical tools that motivated this consolidation; they are shipped as vendored modules and loaded in-process with a thin compatibility layer so that users can continue familiar workflows while benefiting from the unified codebase. Here, NumPy act as the primary array container for inputs and outputs across the analysis and metrics functions (Harris & others, 2020).

Implementation and architecture

Internally, StatMerge keeps analysis and agreement code separate to avoid hidden coupling. Agreement utilities are self-contained and can be imported independently of the GUI, making it straightforward to use them in notebooks or pipelines. The desktop code is intentionally thin: it delegates computations to the library and focuses on layout and file I/O. The legacy graphical tools are loaded via adapters that ensure missing dependencies do not crash the main application. Where those tools expect a `metrics` module, StatMerge provides a compatibility layer that maps to its own tested implementations. The GUI is implemented with PySide6 but does not expose binding details in the user interface.

The choice to implement Wilson score intervals, Hedges' correction, and the Shrout–Fleiss ICC explicitly is pedagogical as much as practical. These formulas are compact, traceable to the literature, and easy to verify in unit tests. Where high-quality reference implementations exist (e.g., scikit-learn's ROC/PR), StatMerge calls into them to reduce duplication and risk.

Usage examples

A typical scripted workflow computes agreement on raw categorical ratings and then explores classifier diagnostics:

```
from statmerge.metrics_mod.agreement import fleiss_kappa_from_raw, binary_metrics
ratings = [['A','A','B'], ['B','B','B'], ['A','B','B'], ['A','A','A']]
kappa, per_item, marginals = fleiss_kappa_from_raw(ratings)

y_true = [1,1,0,0,1,0,1,0]
y_pred = [1,0,0,0,1,1,1,0]
report = binary_metrics(y_true, y_pred)
```

Educators can instead launch the GUI (`statmerge`) to load a CSV of annotations or predictions, preview distributions, and show the interpretive differences between ROC and precision–recall curves when class imbalance is severe (Davis & Goadrich, 2006). The exported DOCX report is intended for quick lab records rather than publication-ready figures.

Conflict of Interests

The author declares no conflict of interests regarding this work.

References

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Routledge. https://doi.org/10.4324/9780203771587

Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233–240. https://doi.org/10.1145/1143844.1143874

Elin, M. N. Z. (2025). *Towards a quantum-inspired paradigm for organizational psychology and behavior: A conceptual modelling approach*. Deep Science Publishing. https://doi.org/10.70593/978-93-7185-418-4

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, *27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, *76*(5), 378–382. https://doi.org/10.1037/h0031619

Haghighi, S., Jasemi, M., Hessabi, S., & Zolanvari, A. (2018). PyCM: Multiclass confusion matrix library in python. *Journal of Open Source Software*, *3*, 729. https://doi.org/10.21105/joss.00729

Harris, C. R., & others. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hedges, L. V., & Olkin, I. (1985). *Statistical methods for meta-analysis*. Academic Press. https://doi.org/10.1016/C2009-0-03396-0

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Icazatti, A., Abril-Pla, O., Klami, A., & Martin, O. A. (2023). PreliZ: A tool-box for prior elicitation. *Journal of Open Source Software*, *8*, 5499. https://doi.org/10.21105/joss.05499

Kelley, L. Z. (2021). Kalepy: A python package for kernel density estimation, sampling and plotting. *Journal of Open Source Software*, *6*, 2784. https://doi.org/10.21105/joss.02784

Koo, T. K., & Li, M. Y. (2016). A guideline of selecting and reporting intraclass correlation coefficients for reliability research. *Journal of Chiropractic Medicine*, *15*(2), 155–163. https://doi.org/10.1016/j.jcm.2016.02.012

Legrand, N., & Allen, M. (2022). Systole: A python package for cardiac signal synchrony and analysis. *Journal of Open Source Software*, *7*, 3832. https://doi.org/10.21105/joss.03832

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th python in science conference* (pp. 51–56). https://doi.org/10.25080/Majora-92bf1922-00a

Patil, I. (2021). statsExpressions: R package for tidy dataframes and expressions with statistical details. *Journal of Open Source Software*, *6*, 3236. https://doi.org/10.21105/joss.03236

Pedregosa, F., & others. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Shrout, P. E., & Fleiss, J. L. (1979). Intraclass correlations: Uses in assessing rater reliability. *Psychological Bulletin*, *86*(2), 420–428. https://doi.org/10.1037/0033-2909.86.2.420

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, *22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

DRAFT